Remove Deprecated strstreams From C++26

Document #:	P2867R2
Date:	2024-03-18
Project:	Programming Language C++
Audience:	Library Evolution Incubator

Reply-to:	Alisdair Meredith <ameredith1@bloomberg.net>

Contents

§ 1 Abstract

Annex D of the C++ Standard, Compatibility features (D [depr]), maintains a deprecated iostreams facility for `char*` strings in user-managed memory that has limited capabilities that are easily misused. This paper proposes removing this facility from the C++ Standard Library as C++20 and C++23 have provided superior replacement facilities.

§ 2 Revision history

§ R2: March 2024 (Tokyo meeting)

- Integrate editorial feedback
- Wording updates
  - Rebased wording onto latest working draft, [N4971]
  - Acknowledge import of library header modules in Annex C
  - Do not try to raionalize why strtreams were bad, just that they were always deprecated
  - “will not compile” -> “may become ill-formed”

§ R1: September 2023 (midterm mailing)

- Removed revision history’s redundant subsection numbering
- Provided complete wording based on the latest working draft, [N4964]
  - Removed the whole of D.13 [depr.str.strstreams]
  - Removed `<strstream>` from the table of all standard headers
  - Added necessary additional names to 16.4.5.3.2 [zombie.names]
  - Supplied Annex C wording
  - Updated stable label cross-reference to C++23
- Recommended closing all open LWG issues on the removed feature

§ R0: Varna 2023

Original version of this document, extracted from the C++23 proposal [P2139R2].

Key changes since that earlier paper

- Rebased wording onto N4944

§ 3 Introduction

At the start of the C++23 cycle, [P2139R2] tried to review each deprecated feature of C++ to see which we would benefit from actively removing and which might now be better undeprecated. Consolidating all this analysis into one place was intended to ease the (L)EWG

review process but in return gave the author so much feedback that the next revision of that paper was not completed.

For the C++26 cycle, a concise paper, [P2863R1], will track the overall review process, but all changes to the Standard will be pursued through specific papers, decoupling progress from the larger paper so that delays on a single feature do not hold up progress on all.

This paper takes up the deprecated `strstream` facility from the original C++98 Standard, D.13 [depr.str.strstreams].

§ 4 Analysis

§ 4.1 Origin

The `char*` streams were provided, predeprecated, in C++98 and have previously been considered for removal. The underlying principle of previous reviews is that this facility not be removed until suitable replacement functionality to which users can migrate is available in a published standard.

§ 4.2 Replacement facilities

C++20 landed the ability to move strings efficiently out of stringstreams in [P0408R7], and C++23 landed the `spanstream` library [P0448R4], which is a candidate for the replacement functionality, so for C++26, we can seriously consider removing this legacy feature, the largest and oldest deprecated feature in the Standard. The Zombie Names clause provides library vendors the ability to retain support for these libraries long after the Standard has stopped specifying them. Therefore, the preferred recommendation of this paper is to finally remove this library from the C++26 Standard.

There remains the alternative position that, when C++26 ships, this facility will have been a supported shipping part of the C++ Standard for almost 30 years. If we have not made serious moves to remove the library in all that time, maybe we should consider undeprecating and removing the shadow of doubt over any code that reaches for this facility today.

We note that two open issues should be resolved as part of any attempt to undeprecate this facility.

1. [LWG3095] `strstreambuf` refers to nonexistent member of `fpos`, `fpos::offset`
2. [LWG3109] `strstreambuf` is copyable

Both issues could be closed without further work if the `char*` streams are removed.

§ 5 Proposal

Remove the long deprecated `char*` streams from C++26.

§ 6 Wording

Make the following changes to the C++ Working Draft. All wording is relative to [N4971], the latest draft at the time of writing.

§ 6.1 Strike `<strstream>` from the list of standard headers

§ 16.4.2.3 [headers] Headers

2 The C++ standard library provides the *C++ library headers*, shown in Table 24.

Table 24: C++ library headers [tab:headers.cpp]

<algorithm>	<format>	<new>	<stdexcept>
<any>	<forward_list>	<numbers>	<stdfloat>
<array>	<fstream>	<numeric>	<stop_token>
<atomic>	<functional>	<optional>	<streambuf>
<barrier>	<future>	<ostream>	<string>
<bit>	<generator>	<print>	<string_view>
<bitset>	<hazard_pointer>	<queue>	~~<strstream>~~
<charconv>	<initializer_list>	<random>	<syncstream>
<chrono>	<iomanip>	<ranges>	<system_error>
<codecvt>	<ios>	<ratio>	<text_encoding>
<compare>	<iosfwd>	<rcu>	<thread>
<complex>	<iostream>	<regex>	<tuple>
<concepts>	<istream>	<scoped_allocator>	<type_traits>
<condition_variable>	<iterator>	<semaphore>	<typeindex>
<coroutine>	<latch>	<set>	<typeinfo>
<deque>	<limits>	<shared_mutex>	<unordered_map>
<exception>	<linalg>	<source_location>	<unordered_set>
<execution>	<list>		<utility>
<expected>	<locale>	<spanstream>	<valarray>
<filesystem>	<map>	<sstream>	<variant>
<flat_map>	<mdspan>	<stack>	<vector>
<flat_set>	<memory>	<stacktrace>	<version>
	<memory_resource>		
	<mutex>		

§ 6.2 Add new identifiers to 16.4.5.3.2 [zombie.names]

§ 16.4.5.3.2 [zombie.names] Zombie names

1 In namespace `std`, the following names are reserved for previous standardization:

— `auto_ptr`,

— `auto_ptr_ref`,

— `binary_function`,

— …

— `is_literal_type`,

— `is_literal_type_v`,

— `istrstream`,

— `little_endian`

— `mem_fun1_ref_t`,

— `mem_fun1_t`,

— …

— `not1`,

— `not2`,

— `ostrstream`,

— `pointer_safety`

— `pointer_to_binary_function`,

— `pointer_to_unary_function`,

— …

— `return_temporary_buffer`,

— `set_unexpected`,

— `strstream`,

— `strstreambuf`,

— `unary_function`,

— `unary_negate`,

— `uncaught_exception`,

— `undeclare_no_pointers`,

— `undeclare_reachable`, and

— `unexpected_handler`.

2 The following names are reserved as members for previous standardization, and may not be used as a name for object-like macros in portable code:

— `argument_type`,

— `first_argument_type`,

— `io_state`,

— `open_mode`,

— `preferred`,

— `second_argument_type`,

— `seek_dir`, and

— `strict`.

3 The ~~name `stossc` is reserved as a member function~~ following names are reserved as member functions for previous standardization, and may not be used as a name for function-like macros in portable code.

— `freeze`,

— `pcount`, and

— `stossc`

4 The header names `<ccomplex>`, `<ciso646>`, `<cstdalign>`, `<cstdbool>`, ~~and~~ `<ctgmath>`, and `<strstream>` are reserved for previous standardization.

§ 6.3 Update Annex C

§ C.1.X Annex D: compatibility features [diff.cpp23.depr]

Change: Remove header `<strstream>` and all its contents.

Rationale: The header has been deprecated since the original C++ standard; the `<spanstream>` header provides an updated, safer facility. Ongoing support remains at implementer's discretion, exercising freedoms granted by 16.4.5.3.2 [zombie.names].

Effect on original feature: A valid C++ 2023 program `#include`-ing or importing the header may become ill-formed. Code that uses any of the following classes by importing one of the standard library modules may become ill-formed:

— `istrstream`

— `ostrstream`

— `strstream`

— `strstreambuf`

§ 6.4 Strike all of D.13 [depr.str.strstreams] `char*` streams

§ D.13 [depr.str.strstreams] `char*` streams

§ D.13.1 [depr.strstream.syn] Header `<strstream>` synopsis

1 The header `<strstream>` defines types that associate stream buffers with character array objects and assist reading and writing such objects.

```
namespace std {
    class strstreambuf;
    class istrstream;
    class ostrstream;
    class strstream;
}
```

§ D.13.2 [depr.strstreambuf] Class `strstreambuf`

§ D.13.2.1 [depr.strstreambuf.general] General

```
namespace std {
  class strstreambuf : public basic_streambuf<char> {
  public:
    strstreambuf() : strstreambuf(0) {}
    explicit strstreambuf(streamsize alsize_arg);
    strstreambuf(void* (*palloc_arg)(size_t), void (*pfree_arg)(void*));
    strstreambuf(char* gnext_arg, streamsize n, char* pbeg_arg = nullptr);
    strstreambuf(const char* gnext_arg, streamsize n);
    strstreambuf(signed char* gnext_arg, streamsize n,
                 signed char* pbeg_arg = nullptr);
    strstreambuf(const signed char* gnext_arg, streamsize n);
    strstreambuf(unsigned char* gnext_arg, streamsize n,
                 unsigned char* pbeg_arg = nullptr);
    strstreambuf(const unsigned char* gnext_arg, streamsize n);

    virtual ~strstreambuf();

    void  freeze(bool freezefl = true);
    char* str();
    int   pcount();

  protected:
    int_type overflow (int_type c = EOF) override;
    int_type pbackfail(int_type c = EOF) override;
    int_type underflow() override;
    pos_type seekoff(off_type off, ios_base::seekdir way,
                     ios_base::openmode which = ios_base::in | ios_base::out)
        override;
    pos_type seekpos(pos_type sp,
                     ios_base::openmode which = ios_base::in | ios_base::out)
        override;
    streambuf* setbuf(char* s, streamsize n) override;

  private:
     using strstate = T1;                       // exposition only

     static const strstate allocated;           // exposition only
     static const strstate constant;            // exposition only
     static const strstate dynamic;             // exposition only
     static const strstate frozen;              // exposition only
     strstate strmode;                          // exposition only
     streamsize alsize;                         // exposition only
     void* (*palloc)(size_t);                   // exposition only
     void (*pfree)(void*);                      // exposition only
  };
}
```

1 The class `strstreambuf` associates the input sequence, and possibly the output sequence, with an object of some *character* array type, whose elements store arbitrary values. The array object has several attributes.

2 [*Note 1:* For the sake of exposition, these are represented as elements of a bitmask type (indicated here as T1) called strstate. The elements are:]

3 [*Note 2:* For the sake of exposition, the maintained data is presented here as:]

4 Each object of class `strstreambuf` has a *seekable area*, delimited by the pointers `seeklow` and `seekhigh`. If `gnext` is a null pointer, the seekable area is undefined. Otherwise, `seeklow` equals `gbeg` and `seekhigh` is either `pend`, if `pend` is not a null pointer, or `gend`.

§ D.13.2.2 [depr.strstreambuf.cons] `strstreambuf` constructors

```
explicit strstreambuf(streamsize alsize_arg);
```

1 *Effects:* Initializes the base class with `streambuf()`. The postconditions of this function are indicated in Table 147.

Table 147: `strstreambuf(streamsize)` effects [tab:depr.strstreambuf.cons.sz]

Element	Value
`strmode`	`dynamic`
`alsize`	`alsize_arg`
`palloc`	a null pointer
`pfree`	a null pointer

2 *Effects:* Initializes the base class with `streambuf()`. The postconditions of this function are indicated in Table 148.

Table 148: `strstreambuf(void* (*)(size_t), void (*)(void*))` effects

Element	Value
`strmode`	`dynamic`
`alsize`	an unspecified value
`palloc`	`palloc_arg`
`pfree`	`pfree_arg`

```
strstreambuf(char* gnext_arg, streamsize n, char* pbeg_arg = nullptr);
strstreambuf(signed char* gnext_arg, streamsize n,
             signed char* pbeg_arg = nullptr);
strstreambuf(unsigned char* gnext_arg, streamsize n,
             unsigned char* pbeg_arg = nullptr);
```

3 *Effects:* Initializes the base class with `streambuf()`. The postconditions of this function are indicated in Table 149.

Table 149: `strstreambuf(charT*, streamsize, charT*)` effects [tab:depr.strstreambuf.cons.ptr]

Element	Value
`strmode`	0
`alsize`	an unspecified value
`palloc`	a null pointer
`pfree`	a null pointer

4 `gnext_arg` shall point to the first element of an array object whose number of elements `N` is determined as follows:

— If `n > 0`, `N` is `n`.

— If `n == 0`, `N` is `std::strlen(gnext_arg)`.

— If `n < 0`, `N` is `INT_MAX`.[1]

5 If `pbeg_arg` is a null pointer, the function executes:

```
setg(gnext_arg, gnext_arg, gnext_arg + N);
```

6 Otherwise, the function executes:

```
setg(gnext_arg, gnext_arg, pbeg_arg);
setp(pbeg_arg,  pbeg_arg + N);
```

```
strstreambuf(const char* gnext_arg, streamsize n);
strstreambuf(const signed char* gnext_arg, streamsize n);
strstreambuf(const unsigned char* gnext_arg, streamsize n);
```

7 *Effects:* Behaves the same as `strstreambuf((char*)gnext_arg,n)`, except that the constructor also sets constant in `strmode`.

```
virtual ~strstreambuf();
```

8 *Effects:* Destroys an object of class `strstreambuf`. The function frees the dynamically allocated array object only if `(strmode & allocated) != 0` and `(strmode & frozen) == 0`. (D.13.2.4 [depr.strstreambuf.virtuals] describes how a dynamically allocated array object is freed.)

§ D.13.2.3 [depr.strstreambuf.members] Member functions

```
void freeze(bool freezefl = true);
```

1 *Effects:* If `strmode & dynamic` is nonzero, alters the freeze status of the dynamic array object as follows:

— If `freezefl` is `true`, the function sets `frozen` in `strmode`.

— Otherwise, it clears `frozen` in `strmode`.

```
char* str();
```

2 *Effects:* Calls `freeze()`, then returns the beginning pointer for the input sequence, `gbeg`.

3 *Remarks:* The return value can be a null pointer.

```
int pcount() const;
```

4 *Effects:* If the next pointer for the output sequence, `pnext`, is a null pointer, returns zero. Otherwise, returns the current effective length of the array object as the next pointer minus the beginning pointer for the output sequence, `pnext - pbeg`.

§ D.13.2.4 [depr.strstreambuf.virtuals] `strstreambuf` overridden virtual functions

```
int_type overflow(int_type c = EOF) override;
```

1 *Effects:* Appends the character designated by `c` to the output sequence, if possible, in one of two ways:

- If `c != EOF` and if either the output sequence has a write position available or the function makes a write position available (as described below), assigns `c` to `*pnext++`. Returns `(unsigned char)c`.
- If `c == EOF`, there is no character to append. Returns a value other than `EOF`.

2 Returns `EOF` to indicate failure.

3 *Remarks:* The function can alter the number of write positions available as a result of any call.

4 To make a write position available, the function reallocates (or initially allocates) an array object with a sufficient number of elements `n` to hold the current array object (if any), plus at least one additional write position. How many additional write positions are made available is otherwise unspecified. If `palloc` is not a null pointer, the function calls `(*palloc)(n)` to allocate the new dynamic array object. Otherwise, it evaluates the expression `new charT[n]`. In either case, if the allocation fails, the function returns `EOF`. Otherwise, it sets `allocated` in `strmode`.

5 To free a previously existing dynamic array object whose first element address is `p`: If `pfree` is not a null pointer, the function calls `(*pfree)(p)`. Otherwise, it evaluates the expression `delete[]p`.

6 If `(strmode & dynamic) == 0`, or if `(strmode & frozen) != 0`, the function cannot extend the array (reallocate it with greater length) to make a write position available.

7 *Recommended practice:* An implementation should consider `alsize` in making the decision how many additional write positions to make available.

```
int_type pbackfail(int_type c = EOF) override;
```

8 Puts back the character designated by c to the input sequence, if possible, in one of three ways:

— If `c != EOF`, if the input sequence has a putback position available, and if `(char)c == gnext[-1]`, assigns `gnext - 1` to `gnext`.
Returns `c`.

— If c != EOF, if the input sequence has a putback position available, and if `strmode & constant` is zero, assigns `c` to `*--gnext`.
Returns c.

— If `c == EOF` and if the input sequence has a putback position available, assigns `gnext - 1` to `gnext`.
Returns a value other than `EOF`.

9 Returns `EOF` to indicate failure.

10 *Remarks:* If the function can succeed in more than one of these ways, it is unspecified which way is chosen. The function can alter the number of putback positions available as a result of any call.

```
int_type underflow() override;
```

11 *Effects:* Reads a character from the input sequence, if possible, without moving the stream position past it, as follows:

— If the input sequence has a read position available, the function signals success by returning `(unsigned char)*gnext`.

— Otherwise, if the current write next pointer `pnext` is not a null pointer and is greater than the current read end pointer `gend`, makes a *read position* available by assigning to `gend` a value greater than `gnext` and no greater than `pnext`.
Returns `(unsigned char)*gnext.`

12 Returns `EOF` to indicate failure.

13 *Remarks:* The function can alter the number of read positions available as a result of any call.

```
pos_type seekoff(off_type off, seekdir way, openmode which = in | out)
        override;
```

14 *Effects:* Alters the stream position within one of the controlled sequences, if possible, as indicated in Table 150.

Table 150: seekoff positioning [tab:depr.strstreambuf.seekoff.pos]

Conditions	Result
`(which & ios::in) != 0`	positions the input sequence

`(which & ios::out) != 0`	positions the output sequence
`(which & (ios::in | ios::out)) == (ios::in | ios::out)` and either `way == ios::beg` or `way == ios::end`	positions both the input and the output sequences
Otherwise	the positioning operation fails.

15 For a sequence to be positioned, if its next pointer is a null pointer, the positioning operation fails. Otherwise, the function determines `newoff` as indicated in Table 151.

Table 151: newoff values [tab:depr.strstreambuf.seekoff.newoff]

Condition	`newoff` Value
`way == ios::beg`	0
`way == ios::cur`	the next pointer minus the beginning pointer (`xnext - xbeg`).
`way == ios::end`	seekhigh minus the beginning pointer (`seekhigh - xbeg`).

16 If `(newoff + off) < (seeklow - xbeg)` or `(seekhigh - xbeg) < (newoff + off)`, the positioning operation fails. Otherwise, the function assigns `xbeg + newoff + off` to the next pointer `xnext`.

17 *Returns:* `pos_type(newoff)`, constructed from the resultant offset `newoff` (of type `off_type`), that stores the resultant stream position, if possible. If the positioning operation fails, or if the constructed object cannot represent the resultant stream position, the return value is `pos_type(off_type(-1))`.

```
pos_type seekpos(pos_type sp, ios_base::openmode which = ios_base::in |
        ios_base::out) override;
```

18 *Effects:* Alters the stream position within one of the controlled sequences, if possible, to correspond to the stream position stored in `sp` (as described below).

— If `(which & ios::in) != 0`, positions the input sequence.

— If `(which & ios::out) != 0`, positions the output sequence.

— If the function positions neither sequence, the positioning operation fails.

19 For a sequence to be positioned, if its next pointer is a null pointer, the positioning operation fails. Otherwise, the function determines `newoff` from `sp.offset()`:

— If `newoff` is an invalid stream position, has a negative value, or has a value greater than `(seekhigh - seeklow)`, the positioning operation fails

— Otherwise, the function adds `newoff` to the beginning pointer `xbeg` and store the result in the next pointer `xnext`.

20 *Returns:* `pos_type(newoff)`, constructed from the resultant offset `newoff` (of type `off_type`), that stores the resultant stream position, if possible. If the positioning operation fails, or if the constructed object cannot represent the resultant stream position, the return value is `pos_type(off_type(-1))`.

```
streambuf<char>* setbuf(char* s, streamsize n) override;
```

21 *Effects:* Behavior is implementation-defined, except that `setbuf(0, 0)` has no effect.

§ D.13.3 [depr.istrstream] Class `istrstream`

§ D.13.3.1 [depr.istrstream.general] General

```
namespace std {
  class istrstream : public basic_istream<char> {
  public:
    explicit istrstream(const char* s);
    explicit istrstream(char* s);
    istrstream(const char* s, streamsize n);
    istrstream(char* s, streamsize n);
    virtual ~istrstream();

    strstreambuf* rdbuf() const;
    char* str();
  private:
    strstreambuf sb;            // exposition only
  };
}
```

1 The class `istrstream` supports the reading of objects of class `strstreambuf`. It supplies a `strstreambuf` object to control the associated array object. For the sake of exposition, the maintained data is presented here as:

— `sb`, the `strstreambuf` object.

§ D.13.3.2 [depr.istrstream.cons] `istrstream` constructors

```
explicit istrstream(const char* s);
explicit istrstream(char* s);
```

1 *Effects:* Initializes the base class with `istream(&sb)` and `sb` with `strstreambuf(s, 0)`. `s` shall designate the first element of an NTBS.

```
istrstream(const char* s, streamsize n);
istrstream(char* s, streamsize n);
```

2 *Effects:* Initializes the base class with `istream(&sb)` and `sb` with `strstreambuf(s,n)`. `s` shall designate the first element of an array whose length is `n` elements, and `n` shall be greater than zero.

§ D.13.3.3 [depr.istrstream.members] Member functions

```
strstreambuf* rdbuf() const;
```

1 *Returns:* `const_cast<strstreambuf*>(&sb)`.

```
char* str();
```

2 *Returns:* `rdbuf()->str()`.

§ D.13.4 [depr.ostrstream] Class `ostrstream`

§ D.13.4.1 [depr.ostrstream.general] General

```
namespace std {
  class ostrstream : public basic_ostream<char> {
  public:
    ostrstream();
    ostrstream(char* s, int n, ios_base::openmode mode = ios_base::out);
    virtual ~ostrstream();

    strstreambuf* rdbuf() const;
    void freeze(bool freezefl = true);
    char* str();
    int pcount() const;

  private:
    strstreambuf sb;            // exposition only
  };
}
```

1 The class `ostrstream` supports the writing of objects of class `strstreambuf`. It supplies a `strstreambuf` object to control the associated array object. For the sake of exposition, the maintained data is presented here as:

— `sb`, the `strstreambuf` object.

§ D.13.4.2 [depr.ostrstream.cons] ostrstream constructors

```
ostrstream();
```

1 *Effects:* Initializes the base class with `ostream(&sb)` and `sb` with `strstreambuf()`.

```
ostrstream(char* s, int n, ios_base::openmode mode = ios_base::out);
```

2 *Effects:* Initializes the base class with `ostream(&sb)`, and `sb` with one of two constructors:

— If `(mode & app) == 0`, then `s` shall designate the first element of an array of `n` elements. The constructor is `strstreambuf(s, n, s)`.

— If `(mode & app) != 0`, then `s` shall designate the first element of an array of `n` elements that contains an NTBS whose first element is designated by `s`. The constructor is `strstreambuf(s, n, s + std::strlen(s))`.[2]

§ D.13.4.3 [depr.ostrstream.members] Member functions

```
strstreambuf* rdbuf() const;
```

1 *Returns:* `(strstreambuf*)&sb`.

```
void freeze(bool freezefl = true);
```

2 *Effects:* Calls `rdbuf()->freeze(freezefl)`.

```
char* str();
```

3 *Returns:* `rdbuf()->str()`.

```
int pcount() const;
```

4 *Returns:* `rdbuf()->pcount()`.

§ D.13.5 [depr.strstream] Class `strstream`

§ D.13.5.1 [depr.strstream.general] General

```
namespace std {
  class strstream
    : public basic_iostream<char> {
  public:
    // types
    using char_type = char;
    using int_type  = char_traits<char>::int_type;
    using pos_type  = char_traits<char>::pos_type;
    using off_type  = char_traits<char>::off_type;

    // constructors/destructor
    strstream();
    strstream(char* s, int n,
              ios_base::openmode mode = ios_base::in|ios_base::out);
    virtual ~strstream();

    // members
    strstreambuf* rdbuf() const;
    void freeze(bool freezefl = true);
    int pcount() const;
    char* str();

  private:
    strstreambuf sb;            // exposition only
  };
}
```

1 The class `strstream` supports reading and writing from objects of class `strstreambuf`. It supplies a `strstreambuf` object to control the associated array object. For the sake of exposition, the maintained data is presented here as:

— `sb`, the `strstreambuf` object.

§ D.13.5.2 [depr.strstream.cons] `strstream` constructors

```
strstream();
```

1 *Effects:* Initializes the base class with `iostream(&sb)`.

```
strstream(char* s, int n,
          ios_base::openmode mode = ios_base::in|ios_base::out);
```

2 *Effects:* Initializes the base class with `iostream(&sb)`, and `sb` with one of the two constructors:

— If `(mode & app) == 0`, then s shall designate the first element of an array of `n` elements. The constructor is `strstreambuf(s,n,s)`.

— If `(mode & app) != 0`, then s shall designate the first element of an array of `n` elements that contains an NTBS whose first element is designated by `s`. The constructor is `strstreambuf(s,n,s+ std::strlen(s))`.

§ D.13.5.3 [depr.strstream.dest] `strstream` destructor

```
virtual ~strstream();
```

1 *Effects:* Destroys an object of class `strstream`.

§ D.13.5.4 [depr.strstream.oper] `strstream` operations

```
strstreambuf* rdbuf() const;
```

1 *Returns:* `const_cast<strstreambuf*>(&sb)`.

```
void freeze(bool freezefl = true);
```

2 *Effects:* Calls `rdbuf()->freeze(freezefl)`.

```
char* str();
```

3 *Returns:* `rdbuf()->str()`.

```
int pcount() const;
```

4 *Returns:* `rdbuf()->pcount()`.

§ 6.5 Update cross-reference for stable labels for C++23

§ Cross-references from ISO C++ 2023

All clause and subclause labels from ISO C++ 2023 (ISO/IEC 14882:2023, *Programming Languages — C++*) are present in this document, with the exceptions described below.

container.gen.reqmts *see*
container.requirements.general

depr.arith.conv.enum *removed*
depr.codecvt.syn *removed*
depr.default.allocator *removed*
depr.istrstream *removed*
depr.istrstream.cons *removed*
depr.istrstream.general *removed*
depr.istrstream.members *removed*
depr.locale.stdcvt *removed*
depr.locale.stdcvt.general *removed*
depr.locale.stdcvt.req *removed*
depr.ostrstream *removed*
depr.ostrstream.cons *removed*
depr.ostrstream.general *removed*
depr.ostrstream.members *removed*
depr.res.on.required *removed*
depr.string.capacity *removed*
depr.str.strstreams *removed*
depr.strstream *removed*
depr.strstream.cons *removed*
depr.strstream.dest *removed*
depr.strstream.general *removed*
depr.strstream.oper *removed*
depr.strstream.syn *removed*
depr.strstreambuf *removed*
depr.strstreambuf.cons *removed*
depr.strstreambuf.general *removed*
depr.strstreambuf.members *removed*
depr.strstreambuf.virtuals *removed*

mismatch *see* alg.mismatch

§ 6.6 Resolve open library issues

The following library issues should be resolved as NAD as they no longer apply to the C++ Standard due to the removal of the feature.

— [LWG3095] `strstreambuf` refers to nonexistent member of `fpos`, `fpos::offset`

— [LWG3109] `strstreambuf` is copyable

§ 7 Acknowledgements

Thanks to Michael Park for the pandoc-based framework used to transform this document's source from .

Huge thanks to Peter Sommerlad for doing all the hard work to make removal of this tricky legacy facility possible.

Thanks to Lori Hughes for reviewing this paper and providing editorial feedback.

§ 8 References

[LWG3095] Billy O'Neal III. strstreambuf refers to nonexistent member of fpos, fpos::offset.
https://wg21.link/lwg3095

[LWG3109] Jonathan Wakely. strstreambuf is copyable.
https://wg21.link/lwg3109

[N4964] Thomas Köppe. 2023-10-15. Working Draft, Programming Languages — C++.
https://wg21.link/n4964

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++.
https://wg21.link/n4971

[P0408R7] Peter Sommerlad. 2019-07-18. Efficient Access to basic_stringbuf's Buffer.
https://wg21.link/p0408r7

[P0448R4] Peter Sommerlad. 2021-03-01. A strstream replacement using span as buffer.
https://wg21.link/p0448r4

[P2139R2] Alisdair Meredith. 2020-07-15. Reviewing Deprecated Facilities of C++20 for C++23.
https://wg21.link/p2139r2

[P2863R1] Alisdair Meredith. 2023-08-15. Review Annex D for C++26.
https://wg21.link/p2863r1

---

1. ~~The function signature `strlen(const char*)` is declared in `<cstring>` (23.5.3 [cstring.syn]). The macro `INT_MAX` is defined in `<climits>` (17.3.6 [climits.syn]).~~ ↩

2. ~~The function signature `strlen(const char*)` is declared in `<cstring>` (23.5.3 [cstring.syn]).~~ ↩